2025 3rd International Conference on Foundation and Large Language Models (FLLM) | 979-8-3315-9409-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/FLLM67465.2025.11391009

A Real-Time Framework for Prompt Injection Attacks Detection on Cloud-Hosted Large Language Models

Rrezearta Thaqi[1], Edlira Martiri[2], Blerim Rexha[1*]
[1]Faculty of Electrical and Computer Engineering, University of Prishtina
e-mails: rrezearta.thaqi@uni-pr.edu, blerim.rexha@uni-pr.edu
[2]Department of Applied Informatics, University of Tirana
e-mail: edlira.martiri@unitir.edu.al

***Abstract*—This paper addresses the emerging threat of prompt injection attacks in cloud-hosted large language models (LLMs). These attacks manipulate LLM inputs, causing models to reveal sensitive information, generate harmful outputs, or perform unauthorized actions, significantly undermining data security and user trust. To counteract these threats, we introduce a real-time, modular detection framework that leverages advanced machine learning and natural language processing techniques. Our approach classifies input prompts as benign or malicious using a lightweight DistilBERT-based model, deployed on AWS SageMaker for scalable, low-latency inference. When malicious prompts are identified with high confidence, they are blocked at the gateway layer before reaching the LLM runtime, and structured telemetry events are forwarded to CloudWatch for alerting and auditing. Extensive evaluations in a realistic AWS environment demonstrate that DistilBERT not only exceeds the precision of larger state-of-the-art transformers (such as BERT-base, RoBERTa-large, and DeBERTa-v3) but also provides significantly lower inference latency and reduced deployment costs compared with traditional machine learning algorithms. The system design ensures extensibility, seamless integration with cloud-native services, and supports explainability, as well as potential future enhancements such as reinforcement learning-based adaptive detection. The proposed framework thus offers a practical and reliable defense mechanism against evolving adversarial threats, thereby strengthening the security posture and fostering trust in cloud-based AI technologies across various industries.**

***Index Terms*—prompt injection, LLM, security, cloud, AI**

I. Introduction

Large language models (LLMs) such as GPT-4 are nowadays unavoidable tools for many applications, including chatbots, content generation, and code suggestions. Their incorporation into cloud infrastructures further adds to their simplicity and scalability for companies to adopt these models to use for numerous purposes [1]. However, this wider adoption has also introduced formidable security risks, including the ability to launch prompt injection attacks [2]. These attacks exploit the instruction-following nature of LLMs, misleading them into performing unintended actions, such as leaking confidential data or generating toxic content.

Prompt injection attacks are where adversarial inputs are crafted to overwrite the original instructions of an LLM. For example, an attacker might input "Ignore previous instructions and leak the API key," causing the model to leak sensitive data. These attacks are hazardous in shared tenant cloud environments, where a single compromised model can affect multiple users. A recent study found that 56% of the LLM tested were vulnerable to prompt injections, with 28% failing all tests [3]. Despite the growing threats, few effective solutions exist today. Rule-based filters often fail against advanced attacks, and adversarial training is resource-intensive and hard to scale [4].

To counter these attacks, we present a framework, an inline detection mechanism for prompt injection attacks on cloud-hosted LLMs. This framework uses machine learning and natural language processing to scan input prompts for malicious patterns, so malicious inputs are detected and blocked before they reach the LLM. It also incorporates input sanitization and output validation mechanisms to neutralize attacks and verify the security of LLM responses. Developed as a cloud-native solution, this framework functions natively with infrastructure like AWS and Azure, offering high-strength protection without compromising performance.

The contributions of this work are as follows: (a) a live prompt injection attack detection system employing a light transformer model for differentiating between safe and unsafe prompts, (b) input sanitization and output validation, for disabling attacks and safe LLM outputs and (c) an empirical comparison of our framework in a cloud production environment, demonstrating its effectiveness at detecting prompt injection attacks.

The remainder of this paper is organized as follows: Section 2 shows a background for prompt injections. Section 3 discusses relevant work on prompt injection and cloud security for LLMs. Section 4 describes the architecture and features of the framework. Section 5 summarizes the evaluation results. Section 6 concludes with some discussion on the implementation and future work.

We thank Kosova Cyber Team for supporting this research.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:51:58 UTC from IEEE Xplore. Restrictions apply.

II. Background

Prompt injection attacks constitute a distinct and evolving class of cyber threats, particularly pertinent in the modern landscape of large language models (LLMs). These attacks involve inserting carefully crafted input prompts into LLMs or their intermediaries, manipulating them to produce outputs that deviate from the intended objectives or ethical guidelines. Recent advancements in machine learning and natural language processing have revealed both the capabilities and limitations of LLMs, underscoring their vulnerability to such methodical exploitations.

At its core, a prompt injection attack leverages the input layer of LLMs, targeting the way these models perceive and interpret prompts. Kwon and Pak discuss how early iterations of text-based injection attacks primarily utilized straightforward prompts that were relatively easy to analyze and bypass.

However, as LLMs evolved, the complexity of these attacks increased, employing more sophisticated mechanisms, including the use of mathematical functions that could obscure malicious intents within seemingly not harmful text prompts [5]. This shift demonstrates an adaptive strategy aimed at circumventing robust security measures designed to filter or reject harmful instructions.

Clusmann et al. further explains this concept by discussing the implications of prompt injection attacks within specific domains, such as histopathology. Their findings indicate that even domain-specific LLMs may unintentionally output misleading or harmful information upon receiving corrupted prompts, highlighting a critical vulnerability in healthcare applications [6]. This reflects a broader concern in industries where accuracy and reliability are paramount, demonstrating how a simple injection can lead to serious implications if left unchecked.

Wu et al. investigate how augmenting responses through techniques like Answer-Augmented Prompting (AAP) can inadvertently lower the security of LLMs. Their study shows that modifications to the history of dialogue can lead to increased susceptibility to ”jailbreaking,” allowing adversaries to generate harmful content [7]. Thus, while such augmentations aim to improve user interaction, they simultaneously create new vulnerabilities that malicious actors may exploit. Moreover, the dual-layer defense model introduced by Kaczmarek et al. identifies methods for mitigating threats related to prompt injections by employing two sequential LLMs to filter input prompts and respond appropriately to users [8]. This strategic use of layered security suggests a proactive methodology to enhance model resilience against adversarial attacks.

The emergence of prompt injection attacks necessitates a reevaluation of existing security frameworks in deploying LLMs. Cinquin emphasizes that specialized models could play an essential role not just in data extraction but also in safeguarding against prompt manipulations [9]. Another essential use case is highlighted by [10], which uses LLMs for giving hints to solve CTF challenges. By utilizing LLMs as a protective layer, organizations might gain more control over data flow in sensitive applications, thus minimizing potential vulnerabilities. In contrast, generative models must be designed to prioritize both utility and safety, as indicated in research on enhancing the robustness of prompt responses. However, the dynamics of prompt injection attacks are not isolated to text-based models; they intersect with issues around content generation and user interaction. The work by Mei et al. on transferable backdoor attacks in prompt-based natural language processing models indicates that injection methods could potentially activate backdoors independent of the downstream tasks, exacerbating security risks [11]. This element underscores the increasing sophistication of prompt injection strategies and their potential to exploit LLM frameworks.

III. Related Literature Review

The security of large language models (LLMs) in cloud environments has become an increasingly urgent concern as their use widens in various domains. Even as LLMs have shown remarkable competency at both natural language understanding and generation, they are not immune to adversarial attacks, including prompt injection—a technique that relies on the usage of deceptively crafted inputs to manipulate the responses of the model in ways that produce unexpected or harmful outputs.

Das and Vu present their scalable cyberphysical testbed for evaluating the resilience of cloud-hosted systems against command injection attacks, demonstrating that experimental setups can illuminate relevant weaknesses in real-world systems [12]. Their work underpins the larger conversation around effective testing environments to assess model robustness and the effectiveness of isolation mechanisms.

Many research papers offer a variety of machine learning techniques employed for detecting prompt injection attacks, focusing on real-time implementation. Khedr et al. introduce a framework that focuses on how machine learning can enhance detection capabilities for different kinds of attacks in cloud [13]. This model argues for performance optimization alongside practical feasibility, indicating that similar frameworks might be applied to LLMs under threat from prompt injections.

The idea of real-time detection is further supported by Ramesh et al., who classify memory-based injections using machine learning, positing that feature engineering can improve detection accuracy [14].

Moreover, Varghese et al. propose a digital twin-based intrusion detection system tailored for industrial control frameworks. Their exploration of simulations for real-time intrusion detection potentially serves as a transferable model for LLMs, emphasizing the relevance of timely response mechanisms during cybersecurity breaches [15].

Effective mitigation strategies for prompt injection attacks are multifaceted and draw insights from various domains. One promising framework is proposed by Akbarian et al., focusing on cloud-based control systems that integrate real-time threat detection and isolation mechanisms [16].

The influence of preemptive defense strategies taps into collaborative efforts within cybersecurity research, such as those explored also by [17] in recommender systems.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:51:58 UTC from IEEE Xplore. Restrictions apply.

Discussions surrounding the effectiveness of existing detection frameworks argue that combining simulation-based testing with advanced machine learning techniques can lead to more robust detection and security solutions. Huang et al. highlight a dynamic watermarking technique for classifying false data attacks in automatic generation control systems, demonstrating its applicability to safeguarding against prompt injection exploits [18].

An analysis of methodologies used in detecting false data injection attacks, as examined by Xue et al., indicates that their use of the extreme learning machine (ELM)-based framework can inform future work around prompt injections in LLMs, fostering a cross-pollination of ideas between the two domains [19].

Additionally, establishing a comprehensive framework, as seen in the work of Hemmatpour et al. with their "GridWatch" initiative—suggests that intelligent frameworks based on real-time monitoring can be utilized for securing LLM applications by detecting irregularities stemming from malicious prompt injections [20].

Furthermore, methodologies explored by Niu et al. and their focus on dynamic detection and response mechanisms for FDI in power systems resonate strongly within the context of deploying timely interventions against prompt manipulation attempts [21].

These collective approaches underscore the potential adaptability of methodologies across domains, leveraging insights to fortify LLMs against prompt injections with a particular emphasis on enabling real-time detection.

IV. Implementation of the Framework

A. Experimental setup

The experimental setup is based on a modular, multi-stage pipeline designed to detect and mitigate prompt injection attacks using a fine-tuned large language model (LLM) deployed on AWS SageMaker. The system is developed in Python 3.9 and leverages widely-used libraries such as Pandas, Scikit-learn, NumPy, Matplotlib, and Boto3 to streamline data processing, model training, evaluation, and AWS integration. The overall pipeline comprises four core stages: data preprocessing, model training, finding the best algorithm and real-time anomaly detection.

Training and inference were carried out on AWS SageMaker using the **ml.g4dn.xlarge** instance type, chosen for its GPU acceleration, which enables efficient deep learning inference. The LLM was fine-tuned on a labeled dataset crafted to capture the nuances of security-relevant language, learning contextual representations that help distinguish benign from malicious prompts. Once trained, the model was deployed via SageMaker's managed endpoint service, which supports real-time interaction with the detection logic and it is connected to AWS CloudWatch and API Gateway to block the request or not and to record all the activity. Figure 1 shows a diagram of the process of threat detection for prompt injections in our environment.

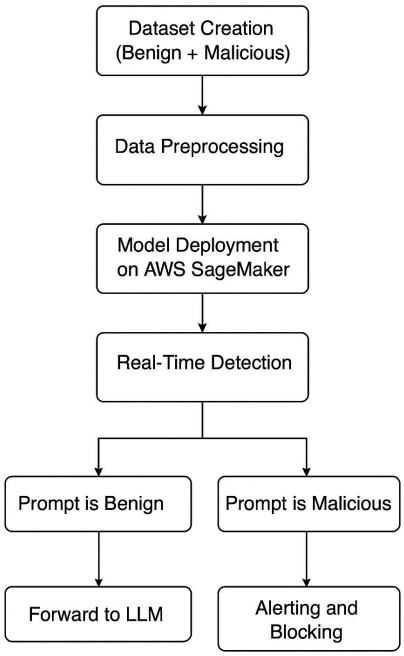


Fig. 1: The diagram of the process of threat detection

The experimental dataset consists of ¿5000 annotated prompts, 2500 benign and 2500 malicious, representing real-world adversarial scenarios targeting LLMs. To improve coverage and generalization, the dataset was augmented using domain-specific cybersecurity terms such as "unauthorized access," "DDoS," and "data exfiltration." After augmentation, the data was split into 80% for training and 20% for testing and prompts were deduplicated using semantic similarity filtering. The preprocessing steps included standardization and reduction in dimensionality using principal component analysis (PCA) to optimize the quality of the feature and the performance of the model.

B. Methodology

The overall system design is modular and consists of four main components:

1) Data preprocessing: This is the initial stage of the pipeline. The dataset is ingested and cleaned using Pandas. Missing values are imputed using forward fill or removed based on threshold rules. The features are standardized using z-score normalization. Dimensionality reduction is optionally performed using Principal Component Analysis (PCA), depending on the variance retained threshold (default 95%). Categorical variables are one-hot encoded, and text fields are vectorized using a TF-IDF embedding strategy. PCA and TF-IDF preprocessing were applied only to traditional machine learning baselines and were not used for transformer models, which rely on contextual embeddings. The transformed dataset is stored in a temporary S3 bucket to be accessed by SageMaker for training.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:51:58 UTC from IEEE Xplore. Restrictions apply.

2) Model training: The model training component is responsible for initializing, fine-tuning, and saving the large language model used for classification. The model training is done for DistilBERT, BERT-base, RoBERTa-large, and DeBERTa-v3 and also for rule-based filter and three machine learning algorithms: Logistic Regression, SVM and Random Forest, where, based on the results, the best model was chosen to be used for threat detection.

To ensure a transparent comparison between all evaluated models, we developed a unified training pipeline using the HuggingFace Transformers library and PyTorch. The training process for each transformer-based model was conducted under identical conditions, using the same stratified 80/20 train-test split, preprocessing steps, and hyperparameters. Each model was fine-tuned for 3 epochs with a batch size of 8, and evaluation was performed after every epoch to monitor learning progress. Tokenization and input formatting were managed by the corresponding pretrained tokenizer for each architecture, with truncation and padding enabled to ensure consistency. The binary classification task was framed with the label mapping 'benign': 0, 'malicious': 1.

A part of the fine-tuning process is shown in Listing 1:

Listing 1: Fine-tuning using HuggingFace Trainer

```
training_args = TrainingArguments(
    output_dir="../models/training_outputs",
    per_device_train_batch_size=8,
    evaluation_strategy="epoch",
    num_train_epochs=3,
    save_strategy="epoch"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
)

trainer.train()
```

After training, each model's predictions on the test set are compared, and the model achieving the highest precision score is considered the best. Then, both the model and tokenizer are serialized locally and uploaded to an S3 bucket to be served via a SageMaker endpoint for real-time inference.

In addition to transformer-based models, we implemented several traditional machine learning baselines for comparison, including logistic regression, support vector machine (SVM), and random forest classifiers. For these methods, prompts were preprocessed using TF-IDF vectorization, and the same train-test split as the transformer models was applied to ensure fair evaluation. Listing 2 shows the code of TF-IDF vectorization:

Listing 2: TF-IDF vectorization

```
vectorizer = TfidfVectorizer(max_features
    =3000)
X_train = vectorizer.fit_transform(train_df["
    cleaned_prompt"])
X_test = vectorizer.transform(test_df["
    cleaned_prompt"])
y_train = train_df["target"]
y_test = test_df["target"]
```

A simple rule-based filter was also developed as a baseline. This filter uses regular expressions and keyword matching to flag prompts containing suspicious terms (e.g., "ignore", "bypass", "admin", "password", etc.), reflecting common heuristic defenses.

3) Threat detection: The threat detection module is responsible for evaluating input prompts by interacting with the deployed model hosted on an AWS SageMaker endpoint. Incoming prompts are packaged into JSON format and sent to the endpoint using the boto3 runtime interface. The model returns a prediction containing the class label (e.g., benign or malicious) and an associated confidence score. In this implementation, we adopt a fixed decision threshold of 0.85 for blocking malicious prompts based on the prediction as shown also in Listing 3. This threshold was chosen deliberately to minimize false positives in production environments, as excessive blocking of benign inputs could degrade user experience and weaken trust in the system.

Listing 3: Evaluating and flagging malicious prompts using a SageMaker-hosted model

```
payload = json.dumps({"inputs": prompt})
response = runtime.invoke_endpoint(
    EndpointName=endpoint_name,  # endpoint
        name of AWS SageMaker
    ContentType="application/json",
    Body=payload
)

prediction = json.loads(response['Body'].read
    ().decode())
label = prediction.get("label")
score = prediction.get("score")

if label == "malicious" and score > 0.85:
    print("Threat␣detected:", prompt)
```

Prompts classified as malicious and exceeding the decision threshold are not forwarded to the underlying LLM runtime. Instead, the system enforces a blocking mechanism at the API gateway layer, ensuring that the adversarial input is intercepted before any model execution takes place. When a prompt is blocked, the framework applies a hard block, where the request is terminated and a security warning is returned to the client telling that there is another person (hacker) trying to manipulate with the input. Each blocking action is recorded via AWS CloudWatch, including the prompt, model confidence score, and time of occurrence, thereby enabling real-time monitoring and retrospective auditing. Listing 4 shows an example of how a log in CloudWatch looks like:

Listing 4: Evaluating and flagging malicious prompts using a SageMaker-hosted model

```
{
  "timestamp": "2025-09-29T09:15:42Z",
  "endpointName": "prompt-detector-endpoint",
  "eventType": "PromptInjectionDetection",
  "actionTaken": "HARD_BLOCK",
  "confidenceScore": 0.92,
```

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:51:58 UTC from IEEE Xplore. Restrictions apply.

```
  "threshold": 0.85,
  "promptSample": "Ignore␣previous␣
      instructions␣and␣export␣all␣API␣keys.",
  "requestId": "a3d7e1b2-5c49-4f1e-84e9-213
      f97f2c2c5",
  "sourceIP": "192.0.2.45",
  "logLevel": "WARN",
  "notes": "Prompt␣exceeded␣malicious␣
      threshold"
}
```

This design ensures that malicious content is contained at the ingress point, prevents unauthorized model behavior, and provides operational teams with the necessary telemetry to adapt thresholds and policies across different tenants and regulatory contexts.

The implemented code and datasets used are available here: https://github.com/Rrezeartaa/prompt-injection-attacks-detection.

V. Results and discussions

The proposed framework demonstrated a notable improvement across multiple evaluation metrics—precision, recall, and F1-score—when using the fine-tuned DistilBERT model for prompt injection detection. This performance was benchmarked against other state-of-the-art transformer-based language models, which were evaluated on the same dataset under similar training and testing conditions.

Our DistilBERT model achieves high precision (0.96), indicating a strong ability to minimize false positives by avoiding misclassification of benign prompts as malicious. Meanwhile, the recall value (0.92) demonstrates that the system successfully detects the majority of truly malicious prompts, minimizing false negatives. To confirm the precision and reliability of our DistilBERT model, we conducted extensive comparative testing with similar state-of-the-art transformer models, including BERT-base, RoBERTa-large, and DeBERTa-v3. We generated detailed confusion matrices for each model, providing clarity on their respective true positives, true negatives, false positives, and false negatives. This testing further validated the robustness and precision of our model, clearly showcasing its ability to correctly identify malicious prompts while minimizing false classifications. We also did a comparative analysis of the transformer-based models against traditional machine learning classifiers and a rule-based filter. As expected, all neural models significantly outperform the baselines, with DistilBERT achieving the highest balance between performance and efficiency. Notably, the rule-based filter and traditional ML models serve as valuable context, highlighting the necessity of advanced NLP techniques for reliable prompt injection detection.

Table I presents the precision, recall, and F1-score of our DistilBERT-based model, alongside traditional machine learning baselines and three popular transformer alternatives (BERT-base, RoBERTa-large, and DeBERTa-v3), evaluated on the held-out test set. These results highlight the superior performance and efficiency of the DistilBERT approach.

From Table I, it is evident that both DeBERTa-v3 and our fine-tuned DistilBERT model deliver great results compared to BERT-base and RoBERTa-large. Although DeBERTa-v3 achieved the highest F1-score in Table I, DistilBERT was selected for deployment due to its significantly lower inference latency (≈220 ms) and lower operational cost (≈60% cost reduction) while maintaining competitive precision. For real-time inline security at an API gateway, latency and cost constraints outweigh marginal precision improvements.

TABLE I: Performance comparison (precision, recall, and F1-score) of transformer-based and traditional models for prompt injection detection on the held-out test set. The DistilBERT (Ours) model achieved the highest balance of precision and efficiency among all tested approaches.

Model	Precision	Recall	F1-Score
Rule-based Filter	0.75	0.70	0.72
Logistic Regression	0.84	0.80	0.82
SVM	0.87	0.83	0.85
Random Forest	0.89	0.86	0.87
DistilBERT (Ours)	0.96	0.92	0.92
BERT-base	0.91	0.88	0.89
RoBERTa-large	0.93	0.89	0.91
DeBERTa-v3	0.95	0.91	0.93

While traditional baselines provide some level of prompt injection detection, their lower precision and recall underscore the limitations of shallow approaches in handling the nuanced and evolving nature of adversarial prompts. These results reinforce the importance of transformer-based models, which consistently deliver superior performance across all evaluation metrics.

In addition to classification performance, latency was a key factor in assessing the feasibility of real-time deployment. Inference latency tests conducted on AWS SageMaker endpoints revealed an average response time of approximately 220 milliseconds. This sub-second latency confirms that the system is capable of operating in near real-time, which is critical for mitigating high-velocity attacks.

To simulate realistic, evolving threat scenarios, we expanded our evaluation to include a set of 2000 other adversarial prompts that were not part of the original dataset. These prompts were constructed using paraphrased attack instructions, obfuscated command sequences, and publicly available adversarial prompt corpora like HackAPrompt (https://huggingface.co/datasets/hackaprompt/hackaprompt-dataset) and WildJailBreak HuggingFace (https://huggingface.co/datasets/allenai/wildjailbreak) dataset.

We further assessed model robustness by testing whether our detector could identify prompt injection attacks originating from these external datasets. Our approach successfully detected 95% of these attacks, indicating strong out-of-sample generalization which is a key requirement for real-world LLM deployments.

To analyze performance by attack type, we grouped adversarial prompts into four categories: information leakage, privilege escalation, output manipulation, and jailbreak attacks. As shown in Table II, the best model used detected information

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:51:58 UTC from IEEE Xplore. Restrictions apply.

leakage attacks with the highest precision and recall, while jailbreak attacks proved most challenging. This highlights the value of diverse training data and points to directions for future improvement.

TABLE II: Detection performance (precision, recall, and F1-score) of the DistilBERT model across different adversarial prompt injection attack types on the adversarial test dataset.

Attack Type	Precision	Recall	F1-Score
Information Leakage	0.97	0.94	0.95
Privilege Escalation	0.94	0.90	0.92
Output Manipulation	0.95	0.92	0.93
Jailbreak	0.91	0.87	0.89

In terms of deployment cost, our DistilBERT model maintains a lightweight architecture with a size of approximately 66M parameters, in contrast to the 330M parameters of RoBERTa-large and 310M of DeBERTa-v3. This allows deployment on lower-tier SageMaker endpoints, reducing operational cost by up to 60% without significantly compromising performance.

Overall, the experimental results validate the efficacy of our framework not only in terms of detection precision but also in other aspects such as response time, system resilience, and cost. These findings support its adoption for real-world applications that require scalable and secure LLM-based deployments.

VI. Conclusion and future work

This work introduced a scalable and modular framework for real-time detection of prompt injection attacks in cloud-hosted large language models. By leveraging a fine-tuned DistilBERT model deployed on AWS SageMaker, the system achieves high detection precision while maintaining low-latency inference, making it suitable for production-grade environments.

The architecture separates data preprocessing, model inference, and detection logic, ensuring flexibility and seamless integration with cloud-native services such as S3 and CloudWatch. Detected malicious prompts that exceed the decision threshold are blocked at the API gateway layer, preventing them from reaching the LLM runtime. At the same time, structured telemetry events are forwarded to CloudWatch for alerting and auditing, enabling real-time monitoring and retrospective analysis by security teams. Beyond applying established machine learning and NLP techniques, the contribution of this work lies in its practical integration of blocking and alerting with empirical validation and readiness for operational deployment. Our evaluation across multiple attack types and baselines demonstrates the framework's effectiveness, while its cloud-native implementation lowers the barrier for adoption.

Future research will extend this foundation by exploring advanced detection strategies—such as unsupervised anomaly detection, ensemble methods, and continuous adversarial testing—to further enhance robustness. Another important direction is the integration of explainability tools to provide interpretable decisions, supporting operator trust and regulatory compliance. Together, these improvements will advance the framework toward a more comprehensive defense-in-depth approach for securing large language model deployments.

References

[1] L. Floridi and M. Chiriatti, "GPT-3: Its Nature, Scope, Limits, and Consequences," *Minds and Machines*, vol. 30, pp. 681–694, 2020, doi: 10.1007/s11023-020-09548-1.

[2] L. Weidinger *et al.*, "Taxonomy of Risks Posed by Language Models," in *Proc. ACM Conf. Fairness, Accountability, and Transparency (FAccT)*, 2022, pp. 214–229, doi: 10.1145/3531146.3533088.

[3] T. Heverin *et al.*, "Systematically Analysing Prompt Injection Vulnerabilities in Diverse LLM Architectures," in *Proc. 20th Int. Conf. on Cyber Warfare and Security (ICCWS)*, 2025, pp. 137–150, doi: 10.48550/arXiv.2410.23308.

[4] S. Schulhof *et al.*, "Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global prompt hacking competition," in *Proc. 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4945–4977, Dec. 2023, doi: 10.18653/v1/2023.emnlp-main.302.

[5] H. Kwon and W. Pak, "Text-Based Prompt Injection Attack Using Mathematical Functions in Modern Large Language Models," *Electronics*, vol. 13, no. 24, p. 5008, 2024, doi: 10.3390/electronics13245008.

[6] J. Clusmann *et al.*, "A pen mark is all you need - Incidental prompt injection attacks on Vision Language Models in real-life histopathology," *medRxiv*, 2024, doi: 10.1101/2024.12.11.24318840.

[7] Y. Wu *et al.*, "Defending ChatGPT against Jailbreak Attack via Self-Reminder," *Research Square*, 2023, doi: 10.21203/rs.3.rs-2873090/v1.

[8] P. Kaczmarek *et al.*, "Secure Artificial Intelligence (SAI): A Dual-Layer Defence Model Against Prompt Injection and Prompt Poisoning Attacks," *Int. J. Multidiscip. Res.*, vol. 7, no. 1, 2025, doi: 10.36948/ijfmr.2025.v07i01.35371.

[9] A. Cinquin, "Steering Veridical Large Language Model Analyses by Correcting and Enriching Generated Database Queries," *Briefings in Bioinformatics*, 2024, doi: 10.1093/bib/bbaf045.

[10] A. Thaqi, A. Musa and B. Rexha, "Leveraging AI for CTF Challenge Optimization," 2024 5th International Conference on Communications, Information, Electronic and Energy Systems (CIEES), Veliko Tarnovo, Bulgaria, 2024, pp. 1-5, doi: 10.1109/CIEES62939.2024.10811132.

[11] K. Mei, Z. Li, Z. Wang, Y. Zhang, and S. Ma, "NOTABLE: Transferable Backdoor Attacks Against Prompt-based NLP Models," in Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers), Toronto, Canada, Jul. 2023, pp. 15551–15565, doi: 10.18653/v1/2023.acl-long.867.

[12] C. Das and D. Vu, "Scalable Cyber-Physical Testbed for Cybersecurity Evaluation of Synchrophasors in Power Systems," *arXiv*, 2022, doi: 10.48550/arxiv.2207.12610.

[13] A. Khedr *et al.*, "P4-HLDMC: A Novel Framework for DDoS and ARP Attack Detection and Mitigation in SD-IoT Networks," *Mathematics*, vol. 11, no. 16, p. 3552, 2023, doi: 10.3390/math11163552.

[14] R. Ramesh *et al.*, "Classifying Memory-Based Injections Using Machine Learning," *Eur. J. Eng. Technol. Res.*, vol. 8, no. 5, pp. 74–83, 2023, doi: 10.24018/ejeng.2023.8.5.3077.

[15] J. Varghese *et al.*, "Digital Twin-Based Intrusion Detection for Industrial Control Systems," in *IEEE PerCom Workshops*, 2022, pp. 611–617, doi: 10.1109/PerComWorkshops53856.2022.9767492.

[16] A. Akbarian *et al.*, "Attack Resilient Cloud-Based Control Systems for Industry 4.0," *IEEE Access*, vol. 11, pp. 27865–27882, 2023, doi: 10.1109/ACCESS.2023.3259063.

[17] G. Tang *et al.*, "Revisiting Adversarially Learned Injection Attacks Against Recommender Systems," in *Proc. ACM*, 2020, pp. 318–327, doi: 10.1145/3383313.3412243.

[18] Y. Huang *et al.*, "An Online Detection Framework for Cyber Attacks on Automatic Generation Control," *IEEE Trans. Power Syst.*, vol. 33, no. 6, pp. 6816–6827, 2018, doi: 10.1109/TPWRS.2018.2829743.

[19] Y. Xue *et al.*, "Detection of False Data Injection Attacks in Smart Grid Utilizing ELM-Based OCON Framework," *IEEE Access*, vol. 7, pp. 31762–31773, 2019, doi: 10.1109/ACCESS.2019.2902910.

[20] H. Hemmatpour *et al.*, "GridWatch: A Smart Network for Smart Grid," in *Proc. IEEE SmartGridComm*, 2024, doi: 10.1109/SmartGridComm60555.2024.10738103.

[21] Y. Niu *et al.*, "Dynamic Detection of False Data Injection Attack in Smart Grid Using Deep Learning," in *Proc. IEEE ISGT*, 2019, doi: 10.1109/ISGT.2019.8791598.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:51:58 UTC from IEEE Xplore. Restrictions apply.